

# Vela

## Confidential Execution on Public Chains

Architecture, and what fits inside it

---

Vela Acceleration Season · Week 1

Khushi - Developer Relations & Coordination @Horizen  
8 September 2026

# The idea everything else hangs from

**“Verified execution,  
not verified logic.”**

Vela proves your code ran unmodified on genuine hardware.  
It does not prove your code is correct.

## Vela in one sentence

An off-chain private execution layer, with smart contracts as entrypoint and exitpoint.

1

**Why this category exists at all**

and what it costs to choose a TEE

2

**What you are actually trusting**

attestation, and its honest limits

3

**How one request really works**

encrypted in, attested out

4

**The envelope**

what fits, and what will not

01

**Why this category exists**

# Public chains have no private state

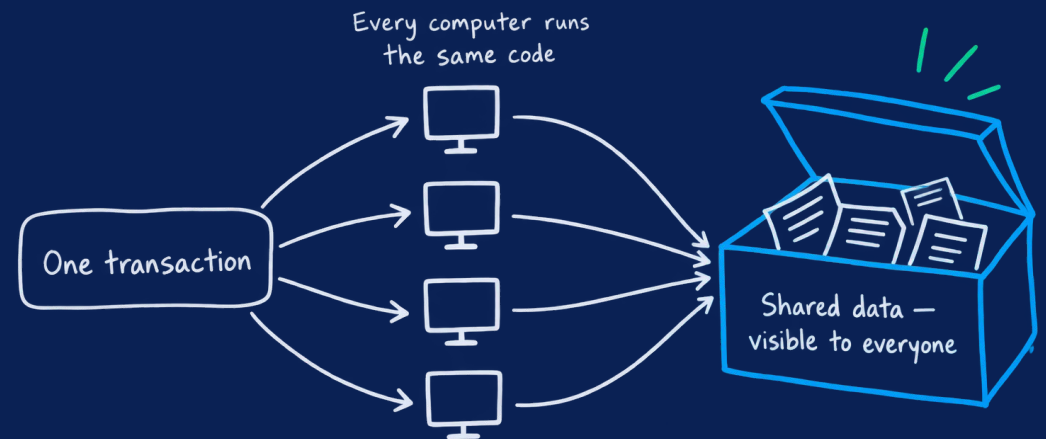
Transparency is not a missing feature — it is the mechanism that makes verification work

## How a blockchain actually verifies

- Every validator re-executes the same bytecode
- ...against the same unencrypted storage
- ...and must reach an identical result
- Anyone can independently check the whole thing

That property is the product. It is also why a credit model, a trading strategy, a patient record or a passport cannot live here.

Everyone checks the same open box



All must agree on the answer

# So why not just use a private server?

Three coordination failures that no amount of good server engineering can solve



## Insider risk

On a server, confidentiality is a policy enforced by access controls. Operators keep root. Policy-based confidentiality collapses the moment there is enough money on the other side of it.



## No composability

An off-chain server cannot escrow or release on-chain assets on its own. The moment it tries, it becomes a custodial intermediary — and you have reinvented the thing you were avoiding.



## No shared custody

When counterparties distrust each other, there is no neutral party to run the server. Whoever hosts it can see everyone's inputs, so nobody agrees to host it.

*Confidential computing exists to solve these three!*

# Four ways to do it

Same goal, very different trade-offs — the comparison investors ask about

|                       | TEE<br>(Vela)                   | ZK                      | FHE                                  | MPC                              |
|-----------------------|---------------------------------|-------------------------|--------------------------------------|----------------------------------|
| Performance profile   | Near-native (~1.1–1.5×)         | High proving overhead   | Very high encrypted-compute overhead | Communication-heavy              |
| Main trust assumption | Hardware / cloud                | Cryptographic soundness | Crypto + key-management model        | Crypto + party / threshold model |
| Programming model     | General code inside the enclave | Circuits, or a zkVM     | Encrypted / oblivious compute        | Distributed protocol             |
| Shared private state  | Natural fit                     | Not from ZK alone       | Possible                             | Via secret sharing               |

TEEs are the closest of these to conventional performance and programming models. ZK carries no hardware trust — but doesn't give you shared private state on its own.

# Where this category actually is today

~\$180k

Oasis Sapphire  
DeFi TVL

~\$894k

Secret Network  
TVL

\$59.5M

Zama  
FHE protocol TVL

~13–15%

Ethereum builder share  
from TEE-based BuilderNet\*

**Confidential application adoption is still early compared with mainstream DeFi, but it is no longer a zero-usage category.**

*Privacy infrastructure works in production. The open question is which confidential application categories develop sustained user demand.*

# Why this is a good time to build

The bet you are making, stated in both directions

## WHAT'S ALREADY TRUE

- The hardware is proven and running in production today
- You can write ordinary code — no rewriting your logic into circuits
- Private state shared between parties is a first-class primitive
- The category is wide open, with no incumbent to unseat

## WORTH KEEPING IN MIND

- You're trusting hardware and a cloud provider, not just maths
- Being early means shaping the space, not just entering it
-

02

**What Vela is,  
and what you're trusting**

# What Vela is — and three things it is not

Clearing up the mental models before we go any deeper

## IT IS

1

### Entrypoint contract

Encrypted request queued, funds locked

2

### WASM app in a Nitro enclave

Decrypt, compute, re-encrypt, sign

3

### Exitpoint contract

Signature verified, events fire, funds move

## IT IS NOT



### Not zero-knowledge

No proofs anywhere. Hardware attestation is a completely different trust model.



### Not a blockchain

It is a coprocessor for the chain you already use. You keep your chain.



### Not FHE or MPC

Plaintext genuinely exists inside the enclave. The isolation is physical, not mathematical.

# The hardware: AWS Nitro, specifically

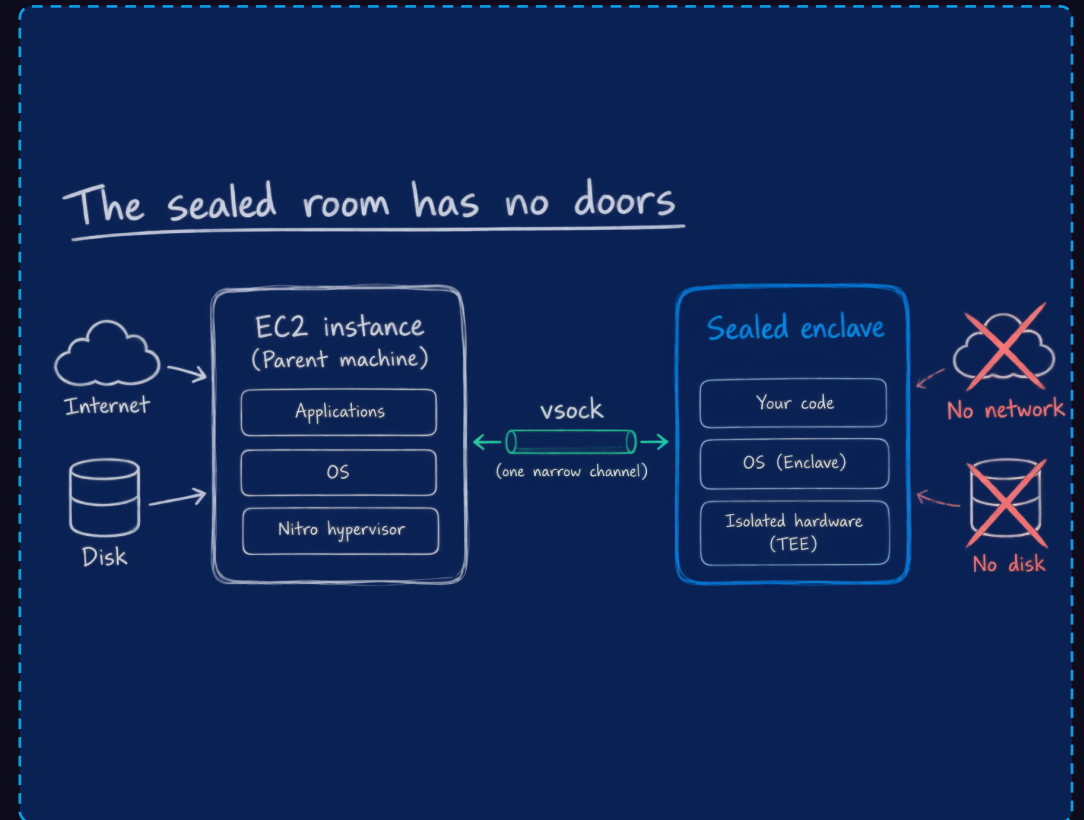
Which TEE Vela uses, and how that quietly changes who you are trusting

## HOW A NITRO ENCLAVE IS ISOLATED

- vCPUs and memory isolated from the parent instance
- No external network connectivity
- No persistent storage
- A single vsock channel to the parent, and nothing else

## The trust root moves

SGX roots trust in an Intel die. Nitro moves it to the AWS Nitro system and the AWS Nitro Attestation PKI — attestation documents are signed by AWS.



# How attestation actually works

From an enclave image measurement to a signature check on-chain

## 1 The image is measured

PCRO is a SHA-384 measurement of the Enclave Image File. AWS separately exposes PCR1 (kernel/bootstrap) and PCR2 (application).

## 2 AWS signs the document

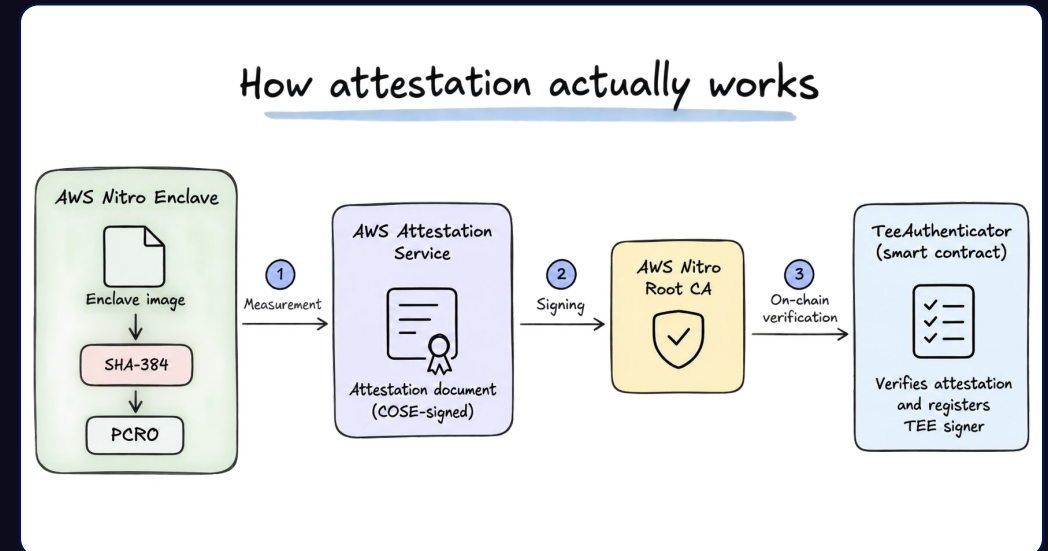
A signed attestation document, chaining back to the AWS Nitro root certificate.

## 3 The chain verifies it

TeeAuthenticator checks the document against an expected PCRO, then registers the enclave's signing address and public key.

## 4 Every result is checked

checkSignature runs inside stateUpdate, recovering the signer and matching it to the registered TEE.



# What you are actually trusting

The honest inventory, including the parts that are uncomfortable



## Attestation is real

PCRO is compared byte-for-byte on-chain and the certificate chain is verified. This part works.



## updatePcr0 is onlyOwner

No timelock or multisig in the contract. The owner decides which enclave image counts — an upgrade authority, visible on-chain but unconstrained.



## A measurement isn't source

PCRO identifies a binary image. Mapping it back to public source needs a reproducible or independently verifiable build.



## AWS is in your trust model

Legal process can reach the cloud account, availability, metadata and data outside the enclave. It shouldn't be described as bypassing Nitro memory isolation.



## It says nothing about your logic

Attestation proves your code ran unmodified. It has no opinion on whether your code is correct.

03

**One request, end to end**

# The pattern almost all of you described

Private inputs + confidential state → a signed result from an attested enclave

**Private inputs + confidential state / parameters → signed result**

Sensitive inputs and application state stay encrypted outside the enclave. On-chain metadata and any intentionally public app events stay visible.

A fit score,  
without the résumé

A diagnosis,  
without the record

A valuation,  
without the comps

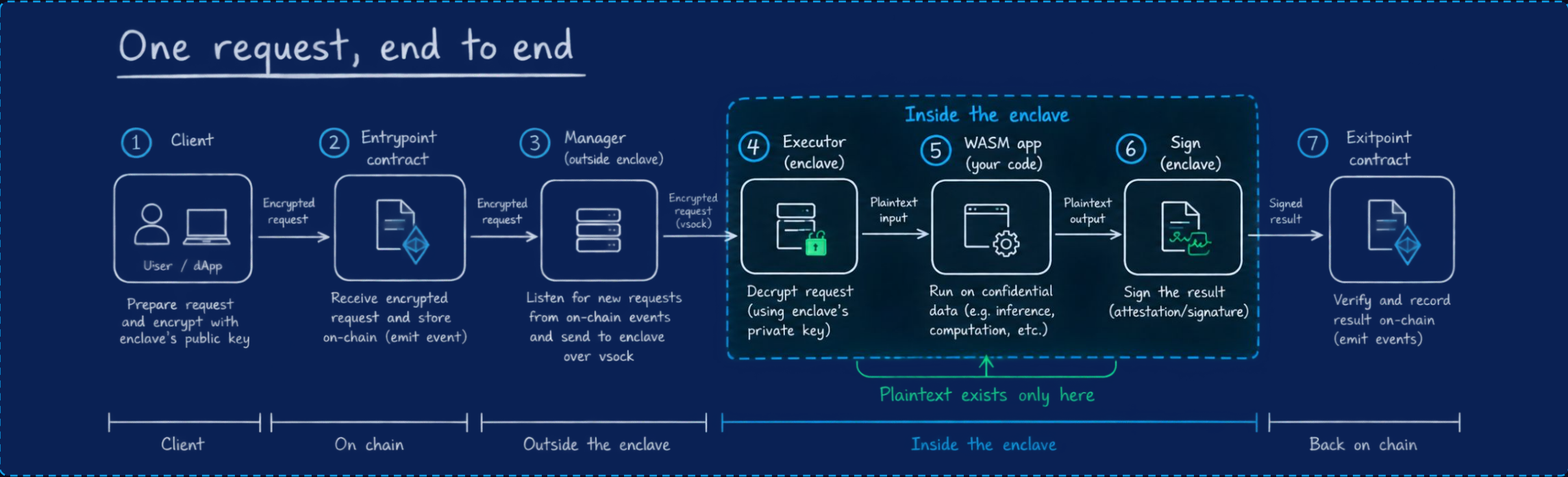
Allow / Hold / Block,  
rules held in state

BID / NO BID,  
without the book

One caveat that changes your design: the deployed WASM binary is loaded from the host and is not confidential from the operator. Keep secret rules and parameters in encrypted state, interpreted by generic code — not hard-coded in the binary.

# One request, all the way through

Seven hops from an encrypted payload to money moving on chain



# Getting a request in

Encrypt to the enclave, submit to the contract, lock the funds

1

## Register a key first

A user publishes their P-521 public key on-chain. Until then, nobody can encrypt anything to them.

2

## Encrypt the payload

ECDH to the enclave's key, through HKDF into AES-GCM. Handled client-side by the TypeScript library.

3

## Submit and lock

The request queues; ETH or an allowlisted ERC-20 locks in the entrypoint contract behind it.

## The five request types

**DEPLOYAPP**

loads your WASM

**PROCESS**

the everyday one

**DEANONYMIZATION**

gated disclosure -  
you design it

**ASSOCIATEKEY**

register a user key

**TRUSTPROCESS**

fired by a trigger

The request payload is opaque. Protocol metadata — sender, app ID, request type, token, amount, fee cap, timing — stays public.

# Inside the enclave

What the Manager can do to you, and what it can never do

**THE MANAGER** · runs outside the enclave

## It CAN

- Censor a request — simply never pass it in
- Delay or reorder the queue
- See who sent what, when, and how much

## It CANNOT

- Read your application state
- Read the contents of a request
- Forge a result — it holds no signing key

## 5 seconds

Default BLOCKCHAIN\_POLLING\_INTERVAL. Your latency floor is this, plus block time, plus the result transaction.

## What runs where

The Executor — inside the enclave — decrypts your state with AES-256-GCM and calls your WASM export: `process_request`, preceded by `deposit` if funds came with the request.

**This split is the whole trust model made concrete: an untrusted component moves the data, a trusted one reads it.**

# Coming back out

The signature check that makes the whole system work, and how you pay for it

## THE CHOKE POINT

The enclave re-encrypts the new state and signs a result payload with secp256k1. On chain, stateUpdate recovers the signer and matches it against the registered TEE address.

The signed message covers the app ID, both state roots, the request ID, hashes of every event and withdrawal, the refund, the fee and the error code.

## Why that exact field list?

Every field in it is something a hostile Manager would otherwise be able to swap between the enclave and the chain. The signature is what nails them all together.

## Paying for it

- Your WASM returns a Fuel value for the work it did
- Fuel is priced at a configurable rate in wei per unit
- Fee plus refund must equal the maxFeeValue the user set
- A MIN\_FEE\_PER\_REQUEST floor is enforced by the contract
- You set each operation's fuel manually, sized to its complexity.

# On chain, off chain, and what leaks anyway

Know exactly what an observer can still see — several plans assume more privacy than this

## ON CHAIN

- Requests and results
- The encrypted state root
- Application IDs
- Funds locked per app
- Events, encrypted per user
- App-level events, in plaintext

## OFF CHAIN

- The full encrypted state
- Your deployed WASM modules
- Deanonymization reports
- Everything the enclave computed along the way

## LEAKS ANYWAY

- The sender's address
- The amount transferred
- The timestamp
- The simple fact that a request happened at all

### One mitigation worth knowing

Event subtypes are indexed log topics, so they are trivial to correlate. If a user registers a subtype seed, the Executor overrides whatever your app supplied with an HMAC-derived value that is unlinkable across users.

04

## The envelope

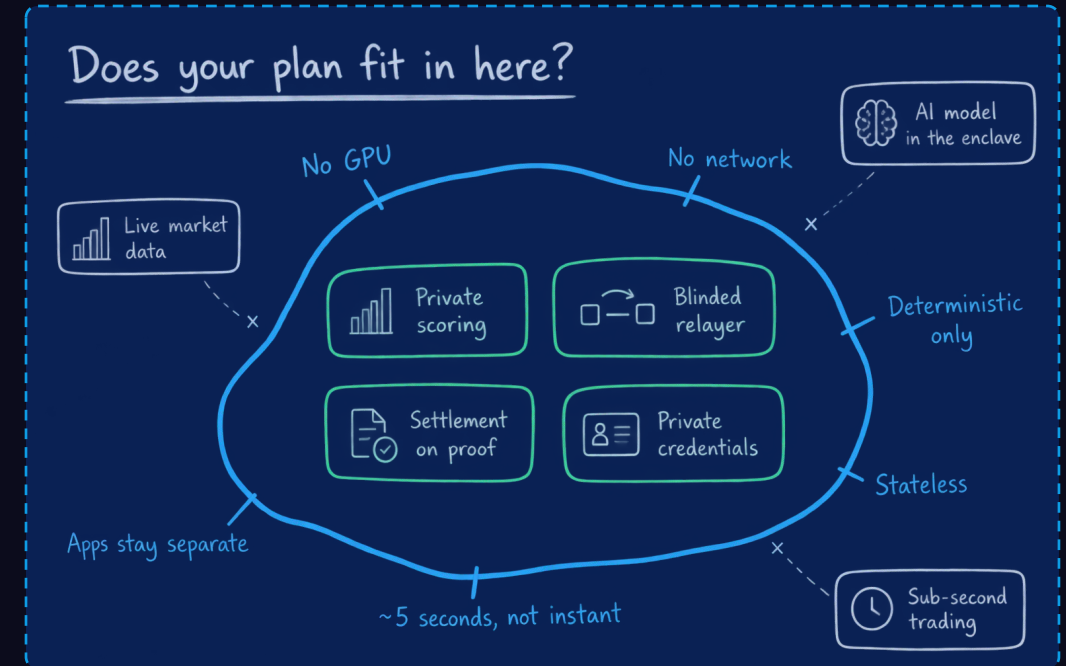
# Your plan has to fit in here

A test to run against the architecture you have already chosen

## You have already decided

Every one of you wrote down what you intend to build with Vela. So the useful thing I can do in the next nine minutes is not hand you ideas ~ it is give you the shape of the box, precisely enough that you can check your own plan against it before writing any code.

**Finding out in week one is the good outcome.**



# The eight constraints you cannot design around

These are properties of the runtime, not gaps waiting to be filled in

## No GPU

Nitro enclaves have none at all

## No network access

No NIC. Data must arrive via the host.

## Must be deterministic

No `time.Now()`, no randomness

## Stateless

your wasm code must be a stateless function, previous saved private state is received as input and must be returned modified as output

## Async, ~5s floor

Polling + block time + result tx

## Apps are isolated

No cross-app calls within a request

## No read-only API

No view equivalent — scan events

## TinyGo limits

No net, most of os, no go-ethereum

# What that rules out, said plainly

Better to hear this now than to discover it in week five



**Fetching live market data inside the enclave**

No network. Something outside has to fetch it and pass it in.



**Training or running a sizeable model in the enclave**

No GPU. In-TEE inference is on the roadmap, not shipped.



**Sub-second trading, liquidations or oracle pushes**

The five-second floor is structural, not a tuning knob.



**One application calling another mid-request**

Applications are isolated from each other by design.



**A synchronous “read my private balance” call**

No view equivalent. Reading means scanning and decrypting events.

# Four shapes that fit comfortably

Everyone in this room should be able to place themselves in one of these

1

## Private scoring and verdicts

Inputs injected, deterministic logic, one attested answer out. The dominant shape in this cohort.

2

## Operator-blind relaying

Take the component that currently sees too much and move just that piece inside the enclave.

3

## Confidential settlement

Funds locked at the entrypoint, released at the exitpoint only against a valid attestation.

4

## Credentials and attested state

Verification logic, plus confidential state that persists across interactions.

05

**Where this goes next**

# Where this is going

What shipped, what is still a proposal, and where it actually deploys

## SHIPPED

- Trigger contracts — private state to public calls
- Standalone token allowlist
- Facilitator with EIP-712 and permit support
- Unlinkable event subtypes

## STILL PROPOSED

- TypeScript runtime instead of WASM
- Inference inside the enclave
- Read-only APIs
- API-key write path
- Private balance handling in the outer layer

## DEPLOYMENT ORDER

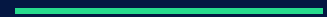
**Base Sepolia & Horizen Testnet**

early access, now

**Base mainnet & Horizen  
Mainnet**

next

# THANK YOU!



*For any questions, ask us on Telegram group!*